# Projeto Final — Módulo 1

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

> **Leia este briefing na semana da aula 09.** Você precisa chegar na aula 10 com o tema escolhido.

## O desafio

Construir uma **aplicação full stack completa**, do banco de dados até a tela, usando tudo que foi visto no módulo.

- **Individual ou em dupla.** Em dupla, os dois precisam saber explicar o código inteiro — a divisão pode ser "eu fiz o back, você fez o front", mas o entendimento tem que ser dos dois.
- **Tema livre**, com aprovação do professor. Escolha algo que resolva um problema **seu**: a motivação faz muita diferença na qualidade final.

---

## Requisitos obrigatórios

Sem estes itens, o projeto não é considerado entregue.

### Back-end

- [ ] API REST em Node.js + Express
- [ ] Pelo menos **um recurso** com CRUD completo (GET lista, GET um, POST, PATCH ou PUT, DELETE)
- [ ] Organizado em **camadas**: routes → controller → service → repository
- [ ] **Validação** de entrada com mensagens claras
- [ ] **Tratamento de erros** centralizado num middleware
- [ ] Status codes corretos (200, 201, 204, 400, 404, 500)
- [ ] Persistência em **SQLite**, com `schema.sql` e script de seed
- [ ] Configuração em `.env`, com `.env.example` versionado
- [ ] CORS habilitado

### Front-end

- [ ] Interface em React com Vite
- [ ] Pelo menos **3 componentes próprios**, além do `App`
- [ ] Consome a **sua** API (nada de dados fixos no código)
- [ ] Lista, cria e remove itens pela interface
- [ ] Estados de **carregando** e de **erro** tratados na tela

- [ ] Formulário com validação e feedback ao usuário
- [ ] Chamadas HTTP centralizadas em `services/api.js`

### Entrega

- [ ] Repositório no GitHub, público
- [ ] `README.md` que permite a qualquer pessoa rodar o projeto
- [ ] `.gitignore` correto — **sem** `node_modules`, `.env` ou `.db` no repositório
- [ ] Histórico com pelo menos **10 commits** com mensagens que descrevem o que mudou
- [ ] Apresentação de 3 a 5 minutos

---

## Temas sugeridos

Se você não tiver uma ideia própria, escolha um destes. Todos cabem no prazo.

| Projeto | Recurso principal | Uma regra de negócio interessante |
|---|---|---|
| **Controle de gastos** | despesas (descrição, valor, categoria, data) | total por categoria e por mês |
| **Diário de treinos** | treinos (exercício, séries, carga, data) | evolução de carga por exercício |
| **Lista de leitura** | livros (título, autor, páginas, status) | páginas lidas por mês |
| **Cardápio semanal** | receitas (nome, ingredientes, tempo, tipo) | sorteio do cardápio da semana |
| **Controle de estoque** | produtos (nome, quantidade, mínimo) | alerta quando cai abaixo do mínimo |
| **Agenda de contatos** | contatos (nome, telefone, e-mail, grupo) | não permitir telefone duplicado |
| **Vagas de emprego** | candidaturas (empresa, cargo, status, data) | funil por status |
| **Hábitos diários** | hábitos + marcações do dia | sequência de dias seguidos |

Independentemente do tema, **inclua pelo menos uma regra de negócio de verdade** — algo além de "salvar e listar". É isso que separa um exercício de uma aplicação.

---

## Prazos

| Momento | O que entregar |
|---|---|
| Aula 09 | Ler este briefing |
| Aula 10 | Chegar com o tema definido e a modelagem rascunhada |
| Aula 10 (durante) | Oficina, com o professor à disposição |
| **1 semana após a aula 10** | Link do repositório entregue |
| A combinar | Apresentação |

## Pontos extras (opcionais)

Só valem se o obrigatório estiver **completo e funcionando**:

| Extra | Peso |
|---|---|
| Um segundo recurso relacionado (com chave estrangeira e `JOIN`) | ★★★ |
| Deploy com URL pública funcionando | ★★★ |
| Filtros, busca e ordenação feitos no servidor | ★★ |
| Tela de estatísticas com dados agregados | ★★ |
| Validação com Zod | ★★ |
| Testes automatizados | ★★★ |
| Interface responsiva (funciona bem no celular) | ★ |
| Modo escuro | ★ |

## O que NÃO é esperado

Para você não gastar energia no lugar errado:

- ❌ Autenticação e login (vem no Módulo 2)
- ❌ Design profissional — funcional e limpo basta
- ❌ Deploy (é bônus, não requisito)
- ❌ TypeScript
- ❌ Bibliotecas de componentes prontos

**O que é avaliado é o que foi ensinado.** Um projeto simples, bem organizado e que funciona sempre vale mais que um projeto ambicioso pela metade.

---

## Arquivos de apoio

| Arquivo | Para quê |
|---|---|
| `CRITERIOS-DE-AVALIACAO.md` | como o projeto será corrigido |
| `TEMPLATE-README.md` | modelo do README do seu repositório |
| `CHECKLIST-DE-ENTREGA.md` | confira item por item antes de entregar |
| `../aula-10/exemplos/starter-fullstack/` | esqueleto funcionando para copiar |
| `../aula-10/exercicios/README.md` | roteiro de trabalho em 5 blocos |

---